Practical Activity Sheet

Case Study & App Development Task: “CampusConnect”

This activity brings together views and layouts, event handling, navigating between screens, Material Design, responsive layout, app-wide navigation, and displaying lists of data into a single running case study and a hands-on Android Studio build.

You will prepare your submission in a separate document (PDF) — see Section E for exactly what it must contain and how to submit it.

Learning Outcomes Addressed

- ILO2 — Choose a suitable mobile development platform and structure based on requirements.
- ILO3 — Design a proper flow of user interfaces for a mobile application.
- ILO4 — Develop and test mobile applications.

Section A — Case Study Scenario

The Department of Industrial Management (DIM) wants a simple mobile app so that department members can log in, see a personalized home screen, browse a list of registered members, and move between the app's main sections without confusion. You have been asked, as the mobile application developer, to build the first working prototype of this app — “CampusConnect”.

“CampusConnect” needs, at minimum:

- A Login screen where a member enters a username and password.
- A Registration screen for new members, reachable from the Login screen and back again.
- A Home screen that greets the logged-in member by name and lets them browse a list of other members.
- A consistent way to move between the app's main sections, with a Logout action that cannot be triggered by a single accidental tap.

Keep this scenario in mind for both Section B (the case study questions) and Section C (the app you will build) — they describe the same app.

Activity Overview

Part	Focus (from the lecture)	What you will produce
Part 1	Views, Layouts & Event Handling	A working Login screen with input validation

Part	Focus (from the lecture)	What you will produce
Part 2	Intents & Passing Data	Navigation between Login, Register and Home, with the username carried across
Part 3	Material Design & Responsive Layout	A Registration screen using Material input fields that holds up on any screen size
Part 4	App-Wide Navigation	A Toolbar with a confirmed Logout, plus Bottom Navigation and/or a Navigation Drawer
Part 5	Displaying Data at Scale	A scrollable list of members using RecyclerView

Section B — Case Study Questions

Answer all five questions below in your own words in your separate submission document (see Section E) — write each answer under its own question heading there. Do not write your answers on this activity sheet. A short, well-explained paragraph is enough for each answer. Based your answers on the CampusConnect scenario in Section A and the concepts from this week's lecture.

Question 1 — Foundations: Views, Layouts & Events

Identify which layout and which views you would use to build the CampusConnect Login screen. Then, using the event-handling flow discussed in the lecture (User Action → Event Generated → Listener Activated → Code Executed → UI Feedback), explain step by step what needs to happen inside the app when a member taps the Login button.

Question 2 — Moving Between Screens: Intents & Data

CampusConnect needs to remember who just logged in as the app moves from the Login screen to the Home screen. Explain how an explicit Intent makes this possible, and why an explicit Intent — rather than an implicit one — is the right tool for navigating within CampusConnect.

Question 3 — Material Design & Responsive Layout

The department wants the CampusConnect Registration form to feel modern and to remain usable on both a small phone and a large tablet. Referring to at least two of Material Design's principles and at least two responsive layout techniques from the lecture, justify the design decisions you would make for this form.

Question 4 — App-Wide Navigation

Once a member logs in, CampusConnect will have three main sections: Home, Members, and Profile. Recommend ONE navigation structure (Bottom Navigation, Tabs, or a Navigation Drawer) for these three

sections and justify your choice. Then explain why the Logout action on the Toolbar should not simply call finish() directly, and what you would use instead.

Question 5 — Scaling the App & Displaying Data

As CampusConnect grows, the department wants to add more sections without slowing the app down or duplicating navigation logic across many Activities. Explain how Fragments address this problem. Then describe the roles that RecyclerView, its Adapter, and its ViewHolder each play in efficiently displaying the list of Members.

Section C — Practical Tasks: Build CampusConnect

You will now build a working prototype of CampusConnect in Android Studio. This section is broken into numbered tasks. Each task states exactly what your app must do and includes one short example showing the general coding pattern involved — but the actual XML and Kotlin for CampusConnect is for you to write. Complete the tasks in order and test your app after each one before moving to the next.

Every task listed here also appears in the task list in Section D — your separate submission document must include a screenshot of the working UI and your code for each one.

Before You Start (Setup — not a numbered task)

1. Open Android Studio and start a New Project.
2. Choose the "Empty Views Activity" template (not a Compose template).
3. **Name:** CampusConnect **Package name:** com.example.campusconnect
4. **Language:** Kotlin **Minimum SDK:** API 24 or the level recommended by your instructor
5. Click Finish and wait for Gradle to finish syncing before you edit anything.
6. Open app/build.gradle (Module) and confirm a line similar to implementation "com.google.android.material:material:..." is already present — the template includes it by default. If it is missing, add it and let Gradle sync again.

Part 1 — Foundations: Build & Wire the Login Screen

The Empty Views Activity template already gave you MainActivity.kt and res/layout/activity_main.xml. We will turn that into the Login screen.

TASK 1 Build the Login Screen Layout

Your task: build res/layout/activity_main.xml so it contains the views listed below.

- An EditText with id etUsername, hint "Username"
- An EditText with id etPassword, hint "Password", with its input type set so the characters are hidden

- A Button with id btnLogin, text “Login”

Example — the general pattern for one EditText inside a layout (adapt the id, hint and input type for each view listed above; this is not the finished screen):

```
<EditText
  android:id="@+id/etExample"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:hint="Example" />
```

TASK 2 Wire Up the Login Button

Your task: in MainActivity.kt, inside onCreate, below setContentView(...), connect the views and validate the input when Login is tapped.

- Get references to all three views using findViewById
- Attach a click listener to the Login button
- Inside the listener, read the text from both fields
- Show a Toast telling the user whether both fields were filled in

Example — the general pattern for responding to a click and showing feedback (you still need to work out the actual condition and messages):

```
exampleButton.setOnClickListener {
  // TODO: read the text out of your EditTexts
  // TODO: decide what the condition should be, then show feedback:
  Toast.makeText(this, "Example message", Toast.LENGTH_SHORT).show()
}
```

If Toast, EditText or Button is underlined in red, place your cursor on it and press Alt+Enter (Option+Return on Mac) to add the missing import automatically.

Check: running the app, tapping Login with both fields empty should show a message telling the user to fill in all fields; typing into both fields and tapping Login should show a success message instead.

Part 2 — Moving Between Screens: Intents & Data

Next, a successful login should take the member to a Home screen that greets them by name.

TASK 3 Create the Home Screen

Your task: create a new HomeActivity with its own layout, ready to greet the member.

1. In the Project panel, right-click the app/java/com.example.campusconnect package → New → Activity → Empty Views Activity.
2. Name it HomeActivity (this also creates res/layout/activity_home.xml and registers it in AndroidManifest.xml automatically).
3. Open activity_home.xml and add a single TextView with id tvWelcome, using the EditText pattern from Task 1 as a guide for the general syntax.

TASK 4 Navigate to Home with the Username

Your task: when login succeeds, open HomeActivity and carry the username across, then display it.

- In MainActivity.kt, on a successful login, create an explicit Intent targeting HomeActivity
- Attach the username the member typed as extra data, under a key you choose (e.g. "USERNAME")
- Start the new Activity
- In HomeActivity.kt, read the extra backout using the same key, and use it to update tvWelcome so it greets the member by name

Example — the general pattern for starting another screen and attaching data to it (the class name, key and value below are placeholders):

```
val intent = Intent(this, ExampleActivity::class.java)
intent.putExtra("EXAMPLE_KEY", "example value")
startActivity(intent)
```

Example — the general pattern for reading an extra back out of an Intent:

```
val value = intent.getStringExtra("EXAMPLE_KEY")
```

Check: logging in with any non-empty username and password should open the Home screen and greet you using the username you typed.

TASK 5 Link Login and Registration Screens

Your task: create a Registration screen and let members move between it and the Login screen.

Repeat Task 3 to create a RegisterActivity with its own activity_register.xml (you will improve this screen further in Part 3). Add a clickable TextView or Button on the Login screen, and use it to open RegisterActivity using the same Intent pattern as Task 4 (without any extra data this time). On the Registration screen, add a similar clickable view that returns to the Login screen.

Example — the general pattern for returning to the previous screen:

```
exampleView.setOnClickListener {
  finish()
}
```

Check: the link from Login to Registration, and back again, should both work.

Part 3 — Material Design & Responsive Layout

Now upgrade the Registration screen so it looks and behaves like a modern Android form, and keeps working on any screen size.

TASK 6 Build the Registration Screen with Material Fields

Your task: open activity_register.xml and build the form using Material input fields.

- A TextInputLayout wrapping a TextInputEditText for Email, with an appropriate hint and input type

- A TextInputLayout wrapping a TextInputEditText for Password, with a password input type and a visibility toggle icon

Example — the general TextInputLayout pattern (adapt the hint, id and input type for each field you need; look back at the lecture slides for the attribute that switches on the password visibility icon):

```
<com.google.android.material.textfield.TextInputLayout
  android:layout_width="match_parent"
  android:layout_height="wrap_content"
  android:hint="Example">

  <com.google.android.material.textfield.TextInputEditText
    android:id="@+id/etExample"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
</com.google.android.material.textfield.TextInputLayout>
```

Check: the password field should show an eye icon that toggles visibility when tapped.

TASK 7 Make the Registration Layout Responsive

Your task: adjust your Registration layout so it still looks correct on any screen size or orientation.

- Keep the root layout as a ConstraintLayout (the default from the template) or a LinearLayout — either is fine, as long as you avoid fixed positioning.
- Use match_parent / wrap_content for widths and heights — never a fixed dp value for something that should stretch.
- Size spacing with dp and text with sp, for example:

```
android:padding="24dp"     <!-- spacing: dp -->
android:textSize="16sp"    <!-- text: sp -->
```

- Wrap your TextInputLayout blocks in a parent layout with some padding, so they stack cleanly on any screen.

Check: rotate the emulator (the rotate icon on the emulator toolbar, or Ctrl+F11 / ⌘+F11) and confirm the form still looks correct and nothing overlaps.

Part 4 — Structuring App-Wide Navigation

With the individual screens working, CampusConnect now needs a consistent way to move around, and a safer way to log out.

TASK 8 Add a Toolbar with a Logout Action

Your task: add a MaterialToolbar to the Home screen with a Logout menu item.

In activity_home.xml, add a MaterialToolbar above your other content, with an id, a title, and a reference to a menu resource you will create. Create the menu resource: right-click res → New → Android Resource File → Resource type: Menu → name it top_menu, and give it one item for Logout.

Example — the general pattern for a Toolbar (choose your own ids and title):

```
<com.google.android.material.appbar.MaterialToolbar
  android:id="@+id/toolbar"
  android:layout_width="match_parent"
  android:layout_height="wrap_content"/>
```

And, separately, in res/menu/top_menu.xml — the general pattern for a menu item:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
  <item android:id="@+id/action_example" android:title="Example"/>
</menu>
```

TASK 9 Confirm Logout with a Dialog

Your task: make sure logging out always asks for confirmation first.

In HomeActivity.kt, detect when your Logout menu item is tapped (the lecture covered setOnMenuItemClickListener for this), and, instead of finishing the Activity straight away, open a dialog asking the member to confirm. Only call finish() from the dialog's positive button.

Example — the general AlertDialog pattern (this is not the Logout dialog itself — write the actual title, message and action yourself):

```
AlertDialog.Builder(this)
  .setTitle("Example Title")
  .setMessage("Example message")
  .setPositiveButton("Yes") { _, _ -> /* your action here */ }
  .setNegativeButton("No", null)
  .show()
```

Make sure finish() is only ever reachable through the dialog's confirmation button — never directly from the menu handler.

Check: tapping Logout should show the dialog; tapping No should do nothing; only confirming should return the app to the Login screen.

TASK 10 Add Bottom Navigation

Your task: give the Home screen a Bottom Navigation bar with three destinations.

Still in activity_home.xml, add a BottomNavigationView below your main content, referencing a new menu resource with three items: Home, Members and Profile. In HomeActivity.kt, respond to taps on each item (a Toast is enough for now).

Example — the general pattern for handling a selection (the item id below is a placeholder):

```
exampleNavView.setOnItemSelectedListener {
  when (it.itemId) {
    R.id.example_item -> true
    else -> false
  }
}
```

Check: tapping each Bottom Navigation item should produce a response.

Bonus Task (optional) — Add a Navigation Drawer

If you have time, wrap the Home screen's content in a DrawerLayout with a NavigationView, following the same structure covered in the lecture, and add 2–3 items to a drawer_menu.xml resource. This bonus task is optional and is not included in the task list in Section D.

A note on Fragments

You do not need to implement Fragments for this activity. Instead, use what you built in Tasks 9 and 10 to answer Case Study Question 5 in Section B — explain, using your Bottom Navigation as a concrete example, where Fragments would replace separate Activities if CampusConnect kept growing.

Part 5 — Displaying Data at Scale: The Members List

Finally, give the Members section of CampusConnect something real to show: a scrollable list of member names.

TASK 11 Build the Member Row Layout and RecyclerView

Your task: create the layout for one row and add a RecyclerView to the Home screen to hold many of them.

Right-click res/layout → New → Layout Resource File → name it item_member, and give it one TextView with id tvMemberName (use the EditText pattern from Task 1 as a guide for the general syntax of a single view in a layout file). Then add a RecyclerView with an id of your choice to activity_home.xml.

Example — the general RecyclerView tag pattern:

```
<androidx.recyclerview.widget.RecyclerView
  android:id="@+id/exampleRecyclerView"
  android:layout_width="match_parent"
  android:layout_height="match_parent"/>
```

TASK 12 Create the Members Adapter

Your task: write a RecyclerView.Adapter that turns a list of names into rows.

Right-click the app package → New → Kotlin Class/File → name it MemberAdapter. It needs to extend RecyclerView.Adapter, define a ViewHolder that references tvMemberName, and implement onCreateViewHolder, onBindViewHolder and getItemCount, as covered in the lecture.

Example — the general shape only (every TODO below is something you must write yourself):

```
class ExampleAdapter(/* your data source here */) :
  RecyclerView.Adapter<ExampleAdapter.ExampleViewHolder>() {

  class ExampleViewHolder(view: View) : RecyclerView.ViewHolder(view) {
    // TODO: find your row's view(s) here
  }

  override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ExampleViewHolder {
    // TODO: inflate item_member and return a ViewHolder wrapping it
  }

  override fun onBindViewHolder(holder: ExampleViewHolder, position: Int) {
    // TODO: set the row's text from your data source at this position
  }

  override fun getItemCount(): Int {
    // TODO: return how many items are in your data source
  }
}
```

TASK 13 Display the Members List

Your task: attach your adapter to RecyclerView so the list actually appears.

In HomeActivity.kt, create a small list of sample member names, then set your RecyclerView's layoutManager and adapter.

Example — the general pattern for attaching a layout manager and adapter (the names below are placeholders):

```
exampleRecyclerView.layoutManager = LinearLayoutManager(this)
exampleRecyclerView.adapter = ExampleAdapter(/* your data */)
```

A short hard-coded list of names is perfectly fine for this activity — the goal is to see RecyclerView, Adapter, and ViewHolder working together.

Check: your sample member names should appear, each in its own row, and the list should scroll smoothly if it is long enough.

Troubleshooting: Common First-Time Issues

Symptom	Likely Cause	Fix
App crashes the instant you tap Login	findViewById id doesn't match the id in the XML	Check the XML android:id and the Kotlin R.id. spelling match exactly, including capitalisation
“Unresolved reference” on a class like Toast, Intent, or AlertDialog	Missing import	Click the red underline and press Alt+Enter (Option+Return on Mac) to auto-import

Symptom	Likely Cause	Fix
Nothing happens when Logout is tapped	The id used in Kotlin doesn't match the id in the menu XML	Check that the id you reference in Kotlin (R.id....) matches the android:id you gave that item in your menu XML exactly
RecyclerView shows a blank screen	layoutManager or adapter was never set	Confirm both recyclerView.layoutManager and recyclerView.adapter are set, in that order, after findViewById
App opens the wrong screen, or crashes with "Activity not found"	New Activity wasn't registered in AndroidManifest.xml	Creating an Activity through New → Activity registers it automatically; if you created the class by hand, add it to AndroidManifest.xml yourself

Section D — Full List of Tasks for Your Submission

Every task from Section C is listed below — all of them must appear in your separate submission document. For each task, create a heading with the task name, and under it include a screenshot of the task's working UI and a screenshot (or copy) of the code you wrote for it.

- ☐ **Task 1 — Build the Login Screen Layout**
- ☐ **Task 2 — Wire Up the Login Button**
- ☐ **Task 3 — Create the Home Screen**
- ☐ **Task 4 — Navigate to Home with the Username**
- ☐ **Task 5 — Link Login and Registration Screens**
- ☐ **Task 6 — Build the Registration Screen with Material Fields**
- ☐ **Task 7 — Make the Registration Layout Responsive**
- ☐ **Task 8 — Add a Toolbar with a Logout Action**
- ☐ **Task 9 — Confirm Logout with a Dialog**
- ☐ **Task 10 — Add Bottom Navigation**
- ☐ **Task 11 — Build the Member Row Layout and RecyclerView**
- ☐ **Task 12 — Create the Members Adapter**
- ☐ **Task 13 — Display the Members List**

Section E — Submission Guidelines

- Prepare your submission in a separate document (PDF).
- In that document, answer all five case study questions from Section B, each written under its own question heading.
- In the same document, for every task listed in Section D, add a heading with the task name, followed by a screenshot of the task's working UI and a screenshot (or copy) of your code for it.
- You do NOT need to upload your Android Studio project or app files to the LMS — only this document is required.
- **Save or export your submission document as a single PDF file.** Name it `YourStudentID_CampusConnect.pdf`.
- Submit only this PDF via the module's LMS page.